Cabal Strat Vault LST Wrapper Security Review

February 24, 2026 - February 27, 2026

Contents

1. About SBSecurity 3

2. Disclaimer 3

3. Risk classification 3

4. Executive Summary 4

5. Findings 5

Low severity 6

[L-01] Off-by-one in lp::get_strat_pending_amount pagination leads to infinite loop and DoS of core entry functions 6
[I-01] Missing zero amount validation in entry function in lp module 8
[I-02] collect_performance_fee() updates high watermark even when fee_shares truncates to zero 9
[I-03] set_performance_fee_bps allows 100% fee rate 9
[I-04] Fee rate changes apply retroactively to all unrealized profit since last watermark update 10
[I-05] Outdated TODO comment suggests unimplemented functionality that actually exists 10
[I-06] Partial cancel can refund xiLP while canceling zero vault shares, creating minor accounting drift 11

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Twitter @Slavcheww.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security-related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1. Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2. Likelihood

- **High** - almost **certain** to happen, easy to perform, or highly incentivized.
- **Medium** - only **conditionally possible**, but still relatively likely.
- **Low** - requires specific state or **little-to-no incentive**.

3.3. Action required for severity levels

- High - **Must** fix (before deployment if not already deployed).
- Medium - **Should** fix.
- Low - **Could** fix.

4. Executive Summary

A time-boxed security review of the **0xRadian/Cabal_stratLP_audit** repository was conducted by **SBSecurity**. The review was performed by **ret2basic.eth**, **p4y4b13** and **radev_eth** who collectively identified **7 issues** in total.

Overview

Project	Cabal Delta-Neutral Vault
Commit Hash	11f8f30f19fd4199998135477b5293364b8ba027
Resolution	83fc75816b0b8d0eda63507bbee92fa69769a660
Timeline	February 24, 2026 - February 27, 2026

Scope

lp.move

Post Audit Condition

The codebase is well-structured and adheres to strong development practices. No Critical or High severity issues were found. Due to the number of recent changes, follow-up audits are recommended, though not mandatory.

5. Findings

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low/Info Risk	7

ID	Title	Severity	Resolution
[L-01]	Off-by-one in `lp::get_strat_pending_amount` pagination leads to infinite loop and DoS of core entry functions	Low	**Fixed**
[I-01]	Missing zero amount validation in entry function in lp module	Info	**Fixed**
[I-02]	`collect_performance_fee()` updates high watermark even when `fee_shares` truncates to zero	Info	**Fixed**
[I-03]	set_performance_fee_bps allows 100% fee rate	Info	Acknowledged
[I-04]	Fee rate changes apply retroactively to all unrealized profit since last watermark update	Info	**Fixed**
[I-05]	Outdated TODO comment suggests unimplemented functionality that actually exists	Info	**Fixed**
[I-06]	Partial cancel can refund xiLP while canceling zero vault shares, creating minor accounting drift	Info	**Fixed**

Low severity

[L-01] Off-by-one in `lp::get_strat_pending_amount` pagination leads to infinite loop and DoS of core entry functions

Severity: Low Risk

Description: `lp::get_strat_pending_amount` is an internal helper that sums all pending vault withdraw request amounts for a asset_store address by paginating through results from `vault::get_user_withdraw_requests` in pages of 100. After each page is processed, the cursor is updated and the loop checks whether to stop:

lp::get_strat_pending_amount

```
fun get_strat_pending_amount(addr: address, collateral_metadata: Object<Metadata>): u64 {
    ...
    let (withdraw_requests_origin, next_key) =
    utils::unpack_pagination_response(page_res_origin);
    key = next_key;
    ...
    // @audit If request are in multiple of 100, The `next_key = None` and the loop continues
    if (withdraw_requests.length() < 100) {
        break;
    };
}
```

When the total number of pending requests is exactly a multiple of 100, the final page returns exactly 100 entries and next_key = None. The break condition evaluates 100 < 100 = false and the loop continues. On the next iteration key is None, the sentinel value for the first page, so the function re-queries from the beginning and repeats indefinitely.

`get_strat_pending_amount` feeds into `get_strat_balance_with_pending`, which is called inside `lp::deposit`, `lp::request_withdraw`, and `lp::withdraw`. Any one of these transactions hitting the infinite loop will exhaust the gas limit and abort, effectively DoS-ing all three entry functions for as long as the condition holds.

Proof of Concept:

1. Arrange for the asset store address to hold exactly 100 (or any multiple of 100) pending withdraw requests in the vault for a given collateral.

2. Call `lp::deposit`, lp::request_withdraw, or lp::withdraw.

3. Execution reaches `get_strat_pending_amount`, which enters the while(true) loop.

4. The last page returns 100 entries with next_key = None. The condition 100 < 100 is false — the loop does not break.

5. The next iteration fires with key = None, restarting from page one. The function never terminates and the transaction is killed by the gas limit.

Recommendation: The loop must also exit when next_key is None, which signals the vault has no further pages regardless of the current page size:

```
fun get_strat_pending_amount(addr: address, collateral_metadata: Object<Metadata>): u64 {
        ...
        let (withdraw_requests_origin, next_key) =
        utils::unpack_pagination_response(page_res_origin);
        key = next_key;
        ...
-       if (withdraw_requests.length() < 100) {
+       if (withdraw_requests.length() < 100 || option::is_none(&key)) {
            break;
        };
    }
```

Resolution: Fixed

[I-01] Missing zero amount validation in entry function in lp module

Severity: Information Risk

Description: The user-facing entry functions `lp::deposit`, `lp::request_withdraw`, and `lp::cancel_withdraw` accept an amount parameter but do not check whether it is greater than zero before proceeding. A caller can submit zero-amount transactions that pass all existing assertions, execute the full function body, emit events, and mutate contract state.

Additionally, when a zero amount is passed to `request_withdraw()`, it creates a phantom entry state in the vault. This request cannot be removed in `cancel_withdraw()` because the VM reverts when dividing by 0. As a result, the request remains stuck until the unlock epoch and is later cleared via `clear_outdated_requests()`.

Impact: Due to this issue, users can create stale phantom entries with zero amounts and spam the strat vault.

Recommendation: Add a zero-amount validation check at the start of each entry function.

Resolution: Fixed

[I-02] `collect_performance_fee()` updates high watermark even when `fee_shares` truncates to zero

Severity: Information Risk

Description: In `collect_performance_fee`, the high watermark update sits outside the `if (fee_shares > 0)` block:

```
if (fee_shares > 0) {
    mint_lp_to(&lp_metadata, m_store.treasury_address, (fee_shares as u64));
};
m_store.high_watermarks.upsert(collateral_metadata, share_price); // ← unconditional
```

When a small price increase causes `fee_shares` to truncate to zero (due to `bigdecimal::mul_by_u128_truncate`), the watermark advances past that gain without collecting any fees. The gain is forever "consumed."

With the default 2% fee and xiLP having 0 decimals, truncation to zero happens for any single-step gain where `gain_ratio × 0.02 / (1 − gain_ratio × 0.02) × supply < 1`. At supply = 10,000, any gain below ~0.5% per interaction yields zero fees.

An malicious user holding a large xiLP position can call any entry function after each small price tick, ratcheting the HWM upward through many zero-fee steps, systematically evading all performance fees. This also happens organically during normal protocol usage. For example frequent user interactions with small inter-transaction price movements cause the treasury to collect nothing.

Impact: Systematic loss of protocol revenue. Performance fees can be partially or fully evaded either deliberately by an attacker or naturally through normal usage patterns.

Recommendation: Only update the HWM when fees are actually minted.

Resolution: Fixed

[I-03] set_performance_fee_bps allows 100% fee rate

Severity: Information Risk

Description: The validation permits `new_bps == BPS_BASE` (10,000 = 100%):

```
assert!(new_bps <= BPS_BASE, error::invalid_argument(EINVALID_FEE_BPS));
```

At 100% fee rate, the dilution formula `fee_shares = supply × d / (1−d)` produces extreme values as `d` approaches 1 for large gains, effectively confiscating nearly all value from xiLP holders through dilution.

Recommendation: Use a strict safety upper bound. (for example - `BPS_BASE / 2`, 50%).

Resolution: Acknowledged

[I-04] Fee rate changes apply retroactively to all unrealized profit since last watermark update

Severity: Information Risk

Description: `collect_performance_fee` uses the current value of `m_store.performance_fee_bps` when computing fees on the profit interval since the last watermark. `set_performance_fee_bps` updates the rate immediately without forcing settlement of existing unrealized profit first. This means all accumulated profit since the last watermark update gets charged at whatever fee rate is active when collection next runs - not the rate that was active when the profit accrued.

Impact: Users or MEV actors can reduce the effective performance fee on already-accrued yield by forcing fee collection before a fee increase, shifting value from the treasury to LP holders.

Recommendation: Call `collect_performance_fee` for all registered collaterals inside `set_performance_fee_bps` before updating the rate.

Resolution: Fixed

[I-05] Outdated TODO comment suggests unimplemented functionality that actually exists

Severity: Information Risk

Description: Directly above `clear_outdated_requests`, there's a TODO comment: "TODO: we need to mint the xiLP tokens back to the user if we clear an outdated request, because we burn when calling request_withdraw"

But the implementation already does exactly this:

```
mint_lp_to(lp_metadata, key.addr, request_data.xilp_amount);
```

The TODO is outdated and misleading. It says that the feature is missing when it is actually implemented.

Recommendation: Remove this TODO.

Resolution: Fixed

[I-06] Partial cancel can refund xiLP while canceling zero vault shares, creating minor accounting drift

Severity: Information Risk

Description: `cancel_withdraw` computes the proportional vault shares to release using integer division:

```
let strat_shares_to_cancel = (((xilp_amount as u128) * (request_data.strat_shares as u128))
/ (request_data.xilp_amount as u128) as u64);
```

When `request_data.strat_shares < request_data.xilp_amount` (normal after fee dilution) and `xilp_amount` is small, `strat_shares_to_cancel` truncates to 0. The function still decrements the local `request_data.xilp_amount`, decrements `pending_withdrawals`, mints xiLP back, and calls `vault::cancel_withdraw(..., 0, false)`.

If the vault treats a 0-share cancel as a no-op, the local state changes but the vault-side pending shares remain locked, creating a small accounting drift.

Impact: Bounded at 1 orphaned share per request key. Vault's epoch expiration clears orphaned shares automatically, so the drift is temporary and economically negligible.

Recommendation: Use ceiling division capped at remaining shares to ensure at least 1 share is always cancelled when xiLP is refunded.

Resolution: Fixed